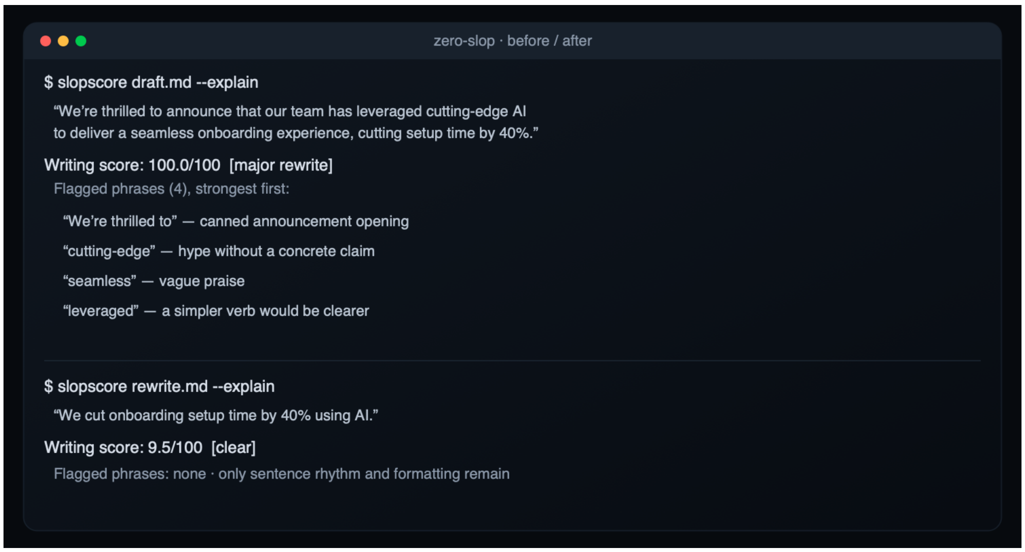
Zero Slop

You used AI to help with a draft, but the result sounds generic. You can hear it, and so can everyone who reads it. That machine sound has a name: slop.

Zero Slop is not an AI model. It runs inside the AI assistant you already use. Claude, GPT, or another compatible model reads and edits the draft; Zero Slop supplies the method and local checks. Its writing score runs from 0 to 100 and points to the phrases and writing patterns that raised it. The assistant then runs separate copy-editing and read-aloud passes. Final checks make sure the facts survived.

```
npx skills add manavmishra/ZeroSlop --global
```

Then open your assistant and say "de-slop this." The Python scorer needs no account or server and does not transmit your writing.



Why it matters now

Sounding like AI can cost credibility, and platforms are adding ways to report, classify, or limit repetitive machine-made material. LinkedIn added an AI-slop report signal; Snap and YouTube apply recommendation or monetization rules to some synthetic or mass-produced content; Substack lets readers scan text for AI signals. Those are different policies, not proof of one universal reach penalty. The practical point is simpler: readers notice generic machine prose, and writers deserve a way to inspect and fix it before publishing.

Who it is for

Anyone who writes under their own name and would rather not sound like a robot. A founder writing a launch post, a marketer shipping five things a week, a researcher who needs their paper to stay formal, an engineering team that wants slop to fail a check the way a bug does.

Eight roles, one workflow

The local scorer finds exact phrases and problems with rhythm, readability, and formatting. The AI assistant acts as interpreter, then rewriter. The local fact gate rejects versions that change names, numbers, quotations, links, code, tables, or document structure. Fresh AI passes act as the copy desk, then the read-aloud editor. Local tools and the assistant share the verifier role, comparing the text with the source. A separate fresh-eyes finalizer reads that verified copy as a first-time reader. These are eight roles, not eight models. If the finalizer changes anything, the last four roles run again. Separating them stops the rewriter from certifying its own work and keeps late edits from bypassing the final checks.

What makes it different

It builds on open-source projects that worked out this craft first: no-ai-slop, humanizer, de-slop, stop-slop, and unslop-text, along with the published work in avoid-ai-writing. It adds three things a plain rewrite lacks.

A score you can inspect. The rules and structural measures are visible, and the same threshold check can run in a build. It is a repeatable writing check, not authorship proof.

A promise about your facts. A rewrite can invent a detail or drop a number to make a sentence flow. Zero Slop lists every figure, name, quote, and link in your original and rejects a version that loses one or adds one. A missing number may be obvious; an invented one can read naturally enough to go unnoticed, which is why the check exists.

A tool that learns from later edits. The editorial loop handles the current draft. A separate private learning loop can compare the assistant's version with a later edit: a phrase you cut may reveal a pattern the local check missed, while flagged text you keep may be a false alarm. No detection rule or preferred fix becomes active after one edit. Before a phrase rule can take effect, the same cut must appear in three unrelated pieces. It must also be new and leave a reference set of human writing unflagged; a single-word cut must appear in five unrelated pieces. A preferred replacement must also recur in three unrelated pieces. The private learning file loads on the next run. Later matching edits confirm useful rules and fixes; without reconfirmation, stale detection rules decay and stale preferred fixes retire.

This is post-deployment, human-in-the-loop online learning. It adapts detection and fixing through inspectable local rules and preferences. Human corrections provide feedback, but Zero Slop does not perform reinforcement learning or RLHF, nor does it retrain the AI model already running in the assistant. Shared changes still require review, tests, versioning, and a release. Each session can check for a newer release with a metadata-only version query. That query never sends the draft.

What we tested

Version 2.6.0 adds four narrow checks for reasoning artifacts, unsupported novelty, emotional flatness, and repetitive acknowledgments, plus the separate fresh-eyes finalizer. The local meter changed none of 114 frozen document scores, kept all 18 known-human controls clear, matched the 84.2 percent result on the 38-item LLM editorial panel, and moved the four new edge cases from 9.5 to 30.7–65.1. Median throughput was 0.94 percent lower in the 12-run local comparison, within the project's five percent regression limit.

We also regenerated four sets of rewrites on the same 18 drafts with GPT-5.4 and the same reasoning level and batch size. Zero Slop passed all of its checks on 18 of 18; avoid-ai-writing passed 15, no-ai-slop 12, and humanizer 9. Because those checks belong to Zero Slop, this is a regression comparison, not independent human field accuracy. The incumbent's own meter preferred its own outputs.

A separate two-pass review hid the method names and reshuffled A/B positions. The GPT-5.4 reviewer favored Zero Slop on 13 drafts and avoid-ai-writing on 3; 2 were unresolved. Both passes chose the same winner on 16 of 18 drafts. Zero Slop's source check passed all 18 of its rewrites and 16 of the incumbent's. This small LLM review does not establish human field accuracy or a universal ranking.

The external distribution checks remain current. RAID+ contributed 7,627 usable generations from four model families; their mean writing scores ranged from 14.5 to 25.5. In Beemo, raw model responses averaged 30.2, expert edits 25.3, and independent human answers 20.0. Neither dataset labels editorial quality. On one Apple silicon Mac, the local checker processed 1,000 documents in 1.9791 seconds, or 505.3 per second. These are reproducible checks with stated limits, not universal claims.
